

S4

Sémantique SQL – chapitre 1

Les sous-requêtes

Une requête dans une requête : trois usages, une distinction à comprendre, et un piège qui vide silencieusement votre résultat.

TU ES ICI SI...

- ▶ « *subquery has too many columns* » ou « *more than one row returned by a subquery* ».
- ▶ « Mon `NOT IN` renvoie zéro ligne, alors que je m'attends à en avoir. »
- ▶ « Je ne sais pas quand utiliser `IN`, `EXISTS`, ou une jointure. »
- ▶ « On me parle de sous-requête *corrélée* et je ne vois pas la différence. »

Une sous-requête est une requête dont le résultat sert d'ingrédient à une autre. Tout dépend de ce qu'elle renvoie : **une valeur** (utilisable dans une comparaison), **une colonne** (utilisable avec `IN`), ou **l'existence de lignes** (avec `EXISTS`). Si elle est corrélée, une sous-requête est ré-exécutée pour chaque ligne de la table étudiée. Sinon, une seule fois.

Trois usages

1. Scalaire : une valeur

```
SELECT title FROM film
WHERE rental_rate >
      (SELECT AVG(rental_rate)
       FROM film);
```

Moyenne = 3,16 → les 3 films à 4,99. Évaluée **une seule fois**, avant le parcours.

2. Ensemble : une colonne

```
SELECT title FROM film
WHERE category_id IN
      (SELECT category_id
       FROM category
       WHERE name = 'Horreur');
```

`IN`, `NOT IN`, `ANY`, `ALL`. La sous-requête doit renvoyer **une seule colonne**.

3. Existence : corrélée

```
SELECT cu.last_name FROM customer cu
WHERE NOT EXISTS
      (SELECT * FROM rental r
       WHERE r.customer_id
             = cu.customer_id);
```

`EXISTS` s'arrête à la première ligne trouvée : ce qu'on y sélectionne n'a aucune importance, d'où le `SELECT *`.

Corrélée ou non ? Le test tient en un coup d'œil

Non corrélée – la sous-requête est autonome. Copiez-la dans une autre fenêtre, elle s'exécute toute seule. Le SGBD l'évalue **une fois**.

```
(SELECT AVG(rental_rate) FROM film)
```

Corrélée – elle mentionne un alias de la requête *externe* (ici `cu`). Elle ne veut rien dire seule, et elle est réévaluée **pour chaque ligne** examinée.

```
(SELECT * FROM rental r WHERE r.customer_id = cu.customer_id)
```

Le test : cherchez dans la sous-requête un alias qui n'y est pas défini. S'il y en a un, elle est corrélée.

Les deux messages d'erreur à reconnaître

CE QUE POSTGRESQL VOUS DIT
 ERROR: subquery has too many columns

Vous avez écrit `IN (SELECT * FROM category WHERE ...)`. Une sous-requête d'un `IN` doit renvoyer **exactement une colonne** : remplacez l'étoile par le nom de la colonne voulue.

CE QUE POSTGRESQL VOUS DIT
 ERROR: more than one row returned by a subquery used as an expression

Vous avez utilisé une sous-requête là où *une valeur* était attendue (après `=`, `>`, ...) et elle en a renvoyé plusieurs. Deux corrections possibles : remplacer `=` par `IN` si plusieurs valeurs sont légitimes, ou ajouter un agrégat (`MAX`, `AVG`...) dans le `SELECT` imbriqué pour n'en avoir qu'une.

Remarque : cette erreur ne survient qu'à l'exécution, et seulement si les données s'y prêtent – une requête qui passe aujourd'hui peut échouer demain.

Sous-requête ou jointure ?

Vous voulez...	Écrivez plutôt
comparer à une valeur globale	une sous-requête scalaire : la jointure ne sait pas exprimer « supérieur à la moyenne ».
afficher des colonnes des deux tables tester une appartenance	une jointure : une sous-requête ne peut rien afficher de son contenu. les deux marchent ; IN/EXISTS ont l'avantage de ne jamais créer de doublons , là où une jointure peut multiplier les lignes.
tester une <i>non</i> -appartenance	NOT EXISTS (sûr), NOT IN (piégeux, voir ci-dessous), ou le motif LEFT JOIN + IS NULL .

LA RÈGLE DE DÉCISION

Trois questions, dans l'ordre :

1. *Ma sous-requête renvoie-t-elle une valeur, une colonne, ou une existence ?* → comparaison, IN, ou EXISTS.
2. *Mentionne-t-elle un alias de l'extérieur ?* → si oui, elle est corrélée, donc réévaluée à chaque ligne.
3. *Est-ce que j'écris une négation ?* → si oui, méfiez-vous du NOT IN et préférez NOT EXISTS.

NOT IN ET LES VALEURS ABSENTES

Supposez que la sous-requête renvoie 1, 2 et un **NULL**. Pour le client 8, SQL doit évaluer :

`8 <> 1 AND 8 <> 2 AND 8 <> NULL = vrai AND vrai AND inconnu = inconnu`

La ligne est donc écartée – et il en va de même pour *toutes* les lignes. **Le résultat est vide, quelles que soient les données.** Aucun message d'erreur.

IN n'a pas ce problème : c'est la *négation* qui le crée. Le réflexe est donc d'écrire NOT EXISTS plutôt que NOT IN dès que la colonne interrogée peut contenir des **NULL**.

VÉRIFIE QUE TU AS COMPRIS

1. Pourquoi WHERE rental_rate > AVG(rental_rate) est-il refusé, alors que WHERE rental_rate > (SELECT AVG(rental_rate) FROM film) fonctionne ?
2. La sous-requête (SELECT MAX(length) FROM film) est-elle corrélée ?
3. « Les films jamais loués » : écrivez-la avec NOT EXISTS.
4. Une sous-requête corrélée est-elle forcément plus lente qu'une jointure ?

RÉPONSES

1. Un agrégat est interdit dans le **WHERE**, qui s'exécute avant tout regroupement (fiche S1) ; la sous-requête, elle, est une requête complète évaluée à part. 2. **Non** : elle ne mentionne aucun alias extérieur, elle s'exécute seule. 3. SELECT f.title FROM film f WHERE NOT EXISTS (SELECT 1 FROM rental r WHERE r.film_id = f.film_id); 4. Pas nécessairement : l'optimiseur réécrit souvent une sous-requête corrélée en jointure. Écrivez d'abord ce qui est *juste et lisible*.